



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 05

NOMBRE COMPLETO: EMILIANO GARCIA OLVERA

N° de Cuenta: 314256009

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

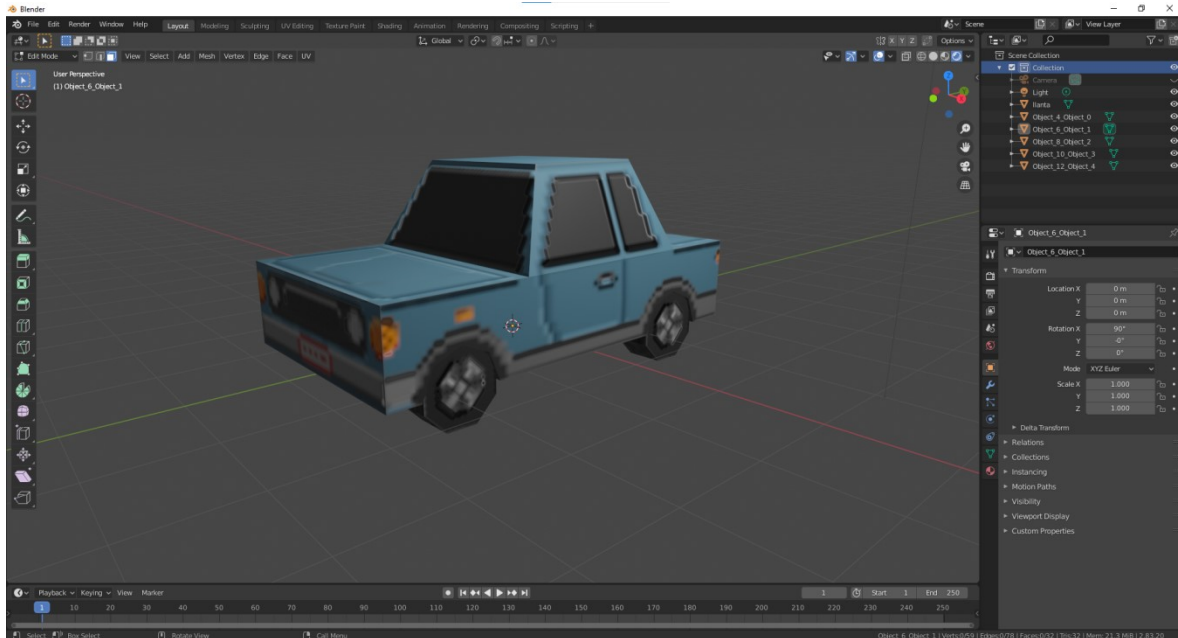
SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 19/MARZO /2026

CALIFICACIÓN: _____

1.- Importar su modelo de coche propio dentro del escenario a una escala adecuada

Para la primera parte de la practica utilizaremos el modelo del auto que conseguimos para el previo.



De esta forma se observa en blender y simplemente queda importarlo en nuestro proyecto OpenGL. Donde primero instanciaremos un tipo Model llamado carro.

`Model` carro;

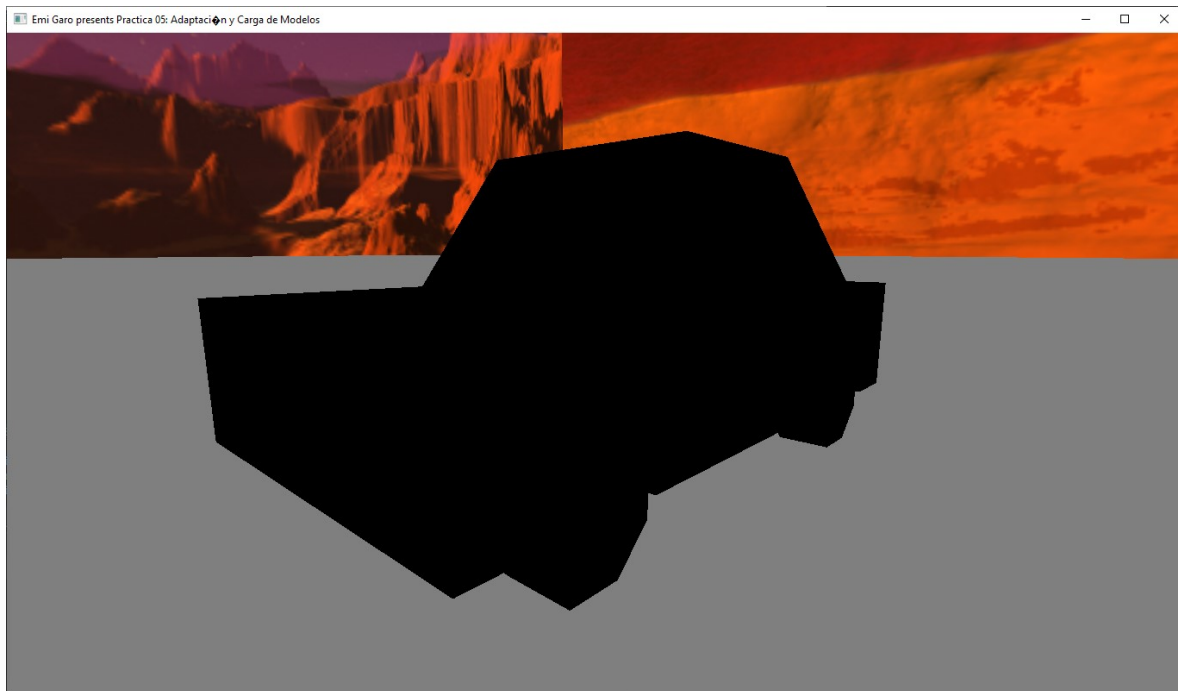
Lo cargaremos con el metodo `LoadModel`, e indicando el path hacia dicho modelo.

```
carro = Model();
carro.LoadModel("Models/carro.obj");
```

Finalmente lo llamaremos en nuestro game loop. Para esto creamos una matriz de identidad pues este sera el nodo padre, lo trasladamos solo en y a 0.3 para que quede encima del piso, guardamos la matriz model y lo renderizamos de color negro.

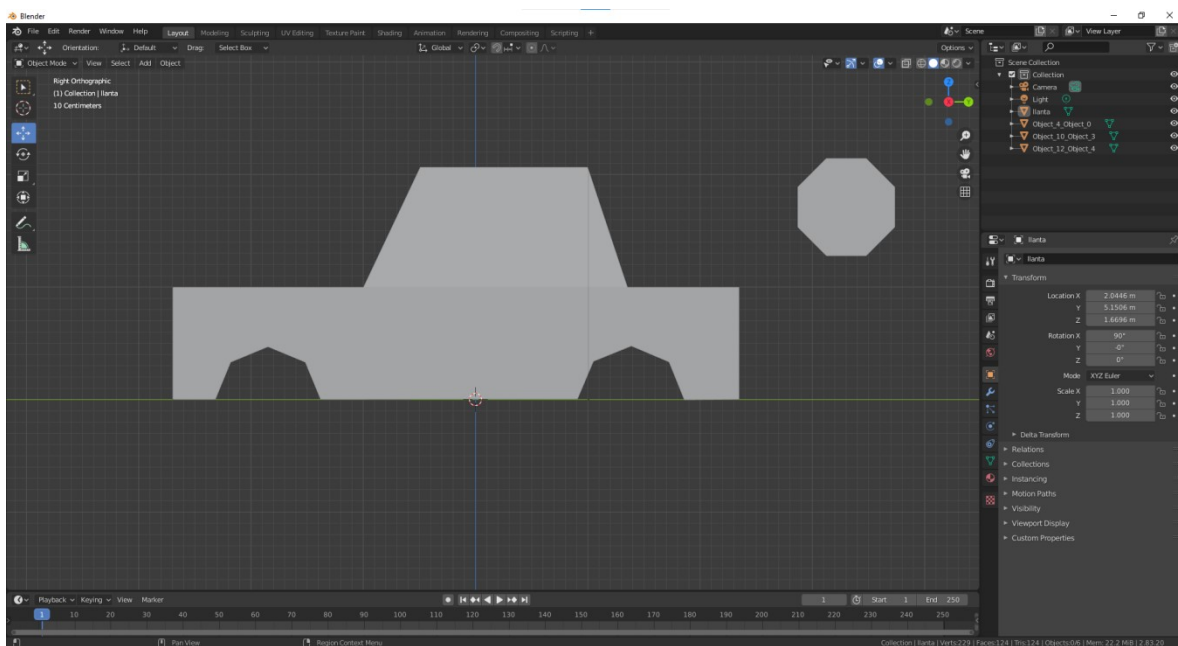
```
// carro
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.3f, 0.0f));
modelaux = model;
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
carro.RenderModel();
```

Y obtenemos este resultado.

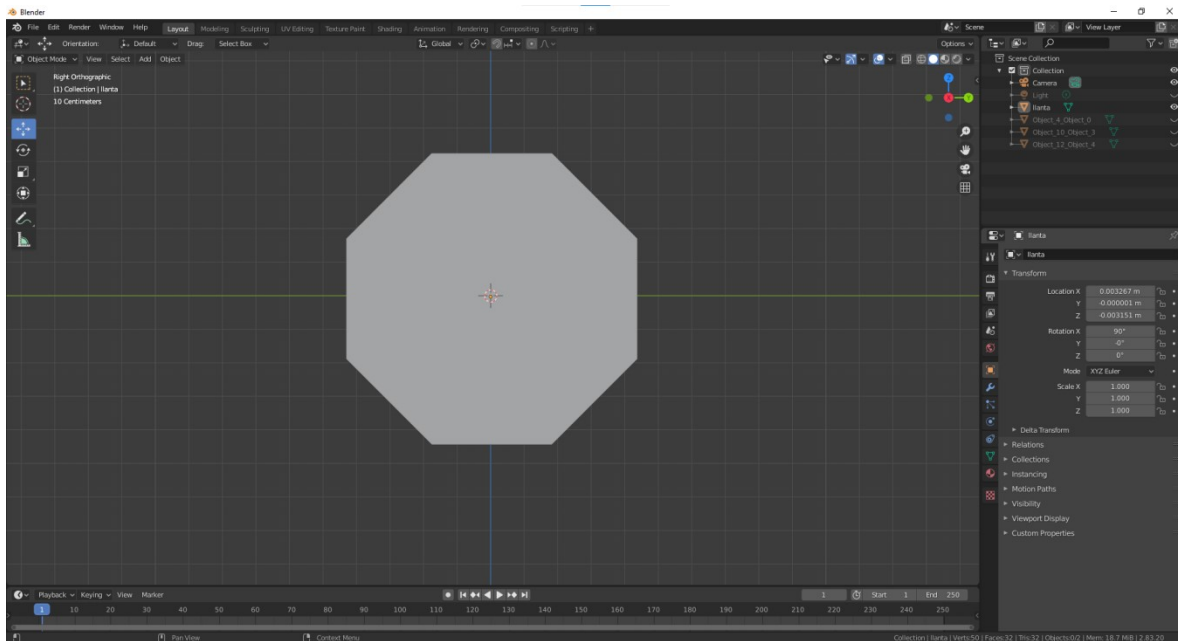


2.- Importar sus 4 llantas y acomodarlas jerárquicamente, agregar el mismo valor de rotación a las llantas para que al presionar puedan rotar hacia adelante y hacia atrás

Para este siguiente ejercicio. Con nuestro modelo cargado en blender, procederemos a separar alguna de las llantas del modelo entero y eliminaremos las demas llantas.



Antes de exportar ambos modelos nos aseguramos que la llanta tenga su centro de origen en el centro de la misma para que al momento de importarlo en el proyecto podamos rotarla alrededor de su eje.



Seguido de este realizamos el mismo procedimiento anterior donde creamos la variable `carro_llanta`, lo cargamos con el modelo exportado de la llanta y la renderizamos cuatro veces (una por llanta). no olvidar que todas las instancias heredan del cuerpo del carro y utilizar `glm::rotate` en el axis x y con los radianes obtenidos de la variable que cambia al precionar F para que las llantas puedan girar.

```
Model carro_llanta;

carro_llanta = Model();
carro_llanta.LoadModel("Models/carro_llanta.obj");

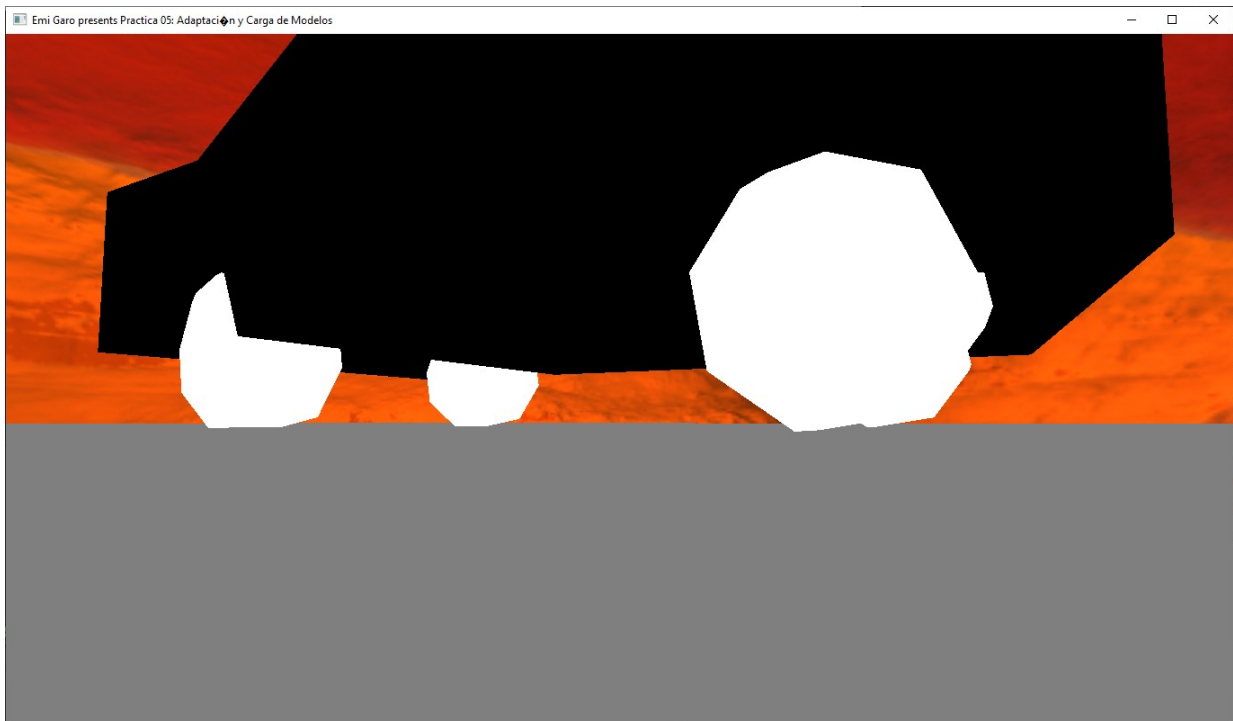
// llanta superior izquierda
model = modelaux;
model = glm::translate(model, glm::vec3(1.0f, 0.0f, 1.9f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()),
glm::vec3(1.0f, 0.0f, 0.0f));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
carro_llanta.RenderModel();
// llanta inferior izquierda
model = modelaux;
model = glm::translate(model, glm::vec3(1.0f, 0.0f, -1.4f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()),
glm::vec3(1.0f, 0.0f, 0.0f));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
carro_llanta.RenderModel();
```

```

// llanta superior derecha
model = modelaux;
model = glm::translate(model, glm::vec3(-1.0f, 0.0f, 1.9f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()),
glm::vec3(1.0f, 0.0f, 0.0f));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
carro_llanta.RenderModel();
// llanta inferior derecha
model = modelaux;
model = glm::translate(model, glm::vec3(-1.0f, 0.0f, -1.4f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()),
glm::vec3(1.0f, 0.0f, 0.0f));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
carro_llanta.RenderModel();

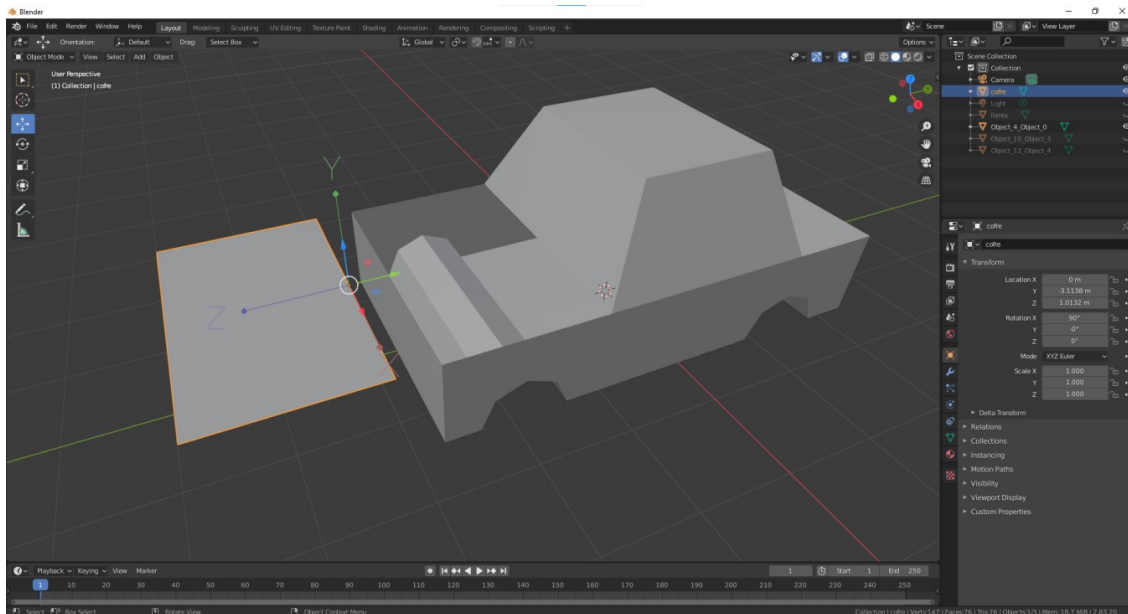
```

Nuestro coche se ve hasta ahora de la siguiente forma.



3.- Importar el cofre del coche, acomodarlo jerárquicamente y agregar la rotación para poder abrir y cerrar.

Para esta parte con el mismo proyecto de blender que hemos estado trabajando, separaremos el cofre del coche de la misma forma que separamos la mandíbula del perro durante el ejercicio de clase. Y no olvidaremos de posicionar su origen de manera que podamos simular el abrir y cerrar del cofre.



De nuevo, mismo procedimiento. Creamos la variable, la damos el path a nuestro modelo exportado y lo posicionamos donde debería estar el cofre.

```
Model carro_cofre;
```

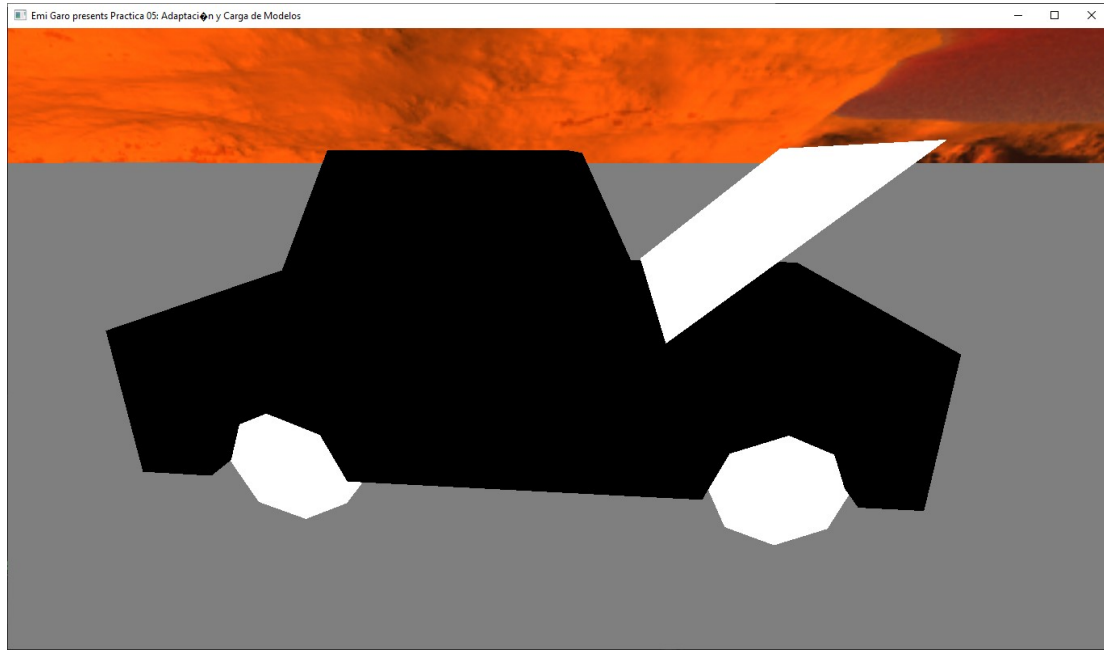
```
carro_cofre = Model();
carro_cofre.LoadModel("Models/carro_cofre.obj");

// cofre
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 1.02f, 1.1f));
model = glm::rotate(model, -glm::radians(mainWindow.getarticulacion2()),
glm::vec3(1.0f, 0.0f, 0.0f));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
carro_cofre.RenderModel();
```

La parte interesante es la siguiente que escribimos en window.cpp

```
if (key == GLFW_KEY_G)
{
    theWindow->articulacion2 += 10.0;
}
if (key == GLFW_KEY_H)
{
    theWindow->articulacion2 -= 10.0;
}
```

Ademas de agregar a articulacion2 (abrir el cofre). Modificamos el evento de la tecla H para restar a articulacion2, emulando el cierre del cofre.



4.- Agregar traslación con teclado para que pueda avanzar y retroceder de forma independiente

Para este ultimo ejercicio ya tenemos todo en cuanto a modelos. Por lo que si trabajamos bien hasta ahora (teniendo el cuerpo de carro como nodo padre e instanciando las partes a partir de este), para simular la traslacion implementaremos el siguiente codigo.

En nuestro header de la ventana (window.h) implementamos la variable position con su respectivo getter, y en su contraparte .cpp inicializamos dicha variable con el valor de 0.0

```
GLfloat position;

GLfloat get_position() { return position; }

position = 0.0f;
```

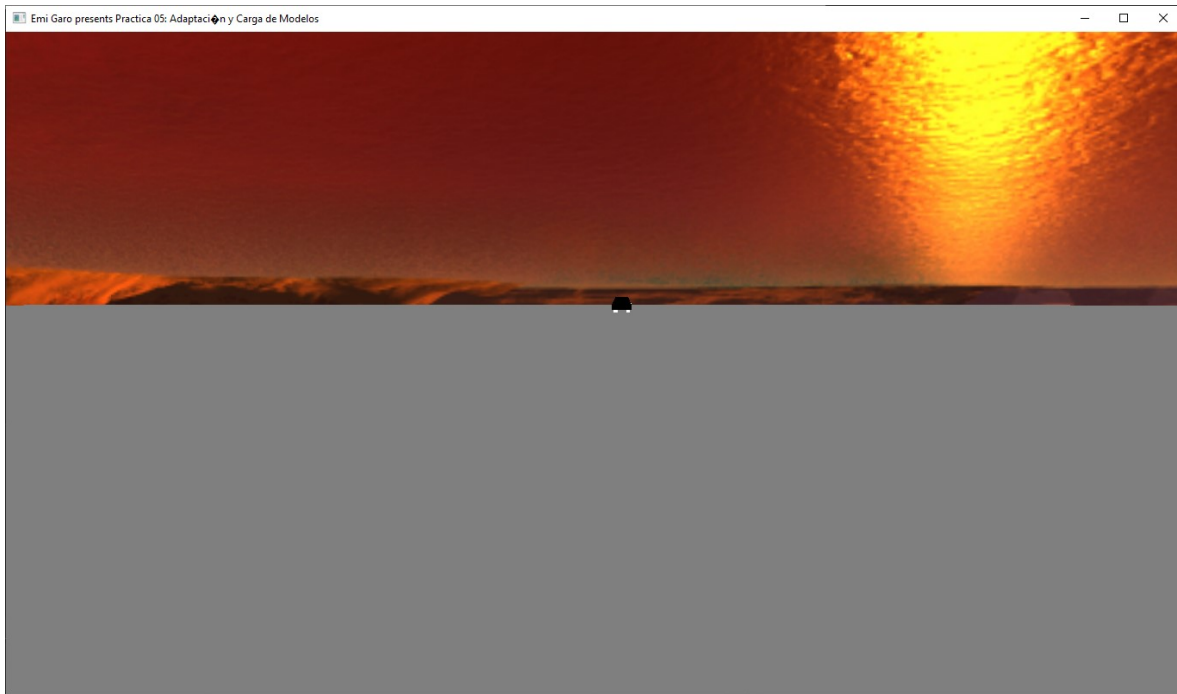
Ya que tenemos nuestra variable para trabajar, agregamos el codigo para que al accionar las teclas up y down agreguemos o restemos a nuestra variable position. Notese que utilizamos un valor relativamente pequeño.

```
if (key == GLFW_KEY_UP)
{
    theWindow->position += 1.0;
}
if (key == GLFW_KEY_DOWN)
{
    theWindow->position -= 1.0;
}
```


Finalmente modificamos el translate del model para que en el eje Z (hacia donde apunta nuestro carro por defecto), este pueda avanzar o retroceder utilizando como argumento nuestra variable position

```
model = glm::translate(model, glm::vec3(0.0f, -1.4f, mainWindow.get_position()));
```

Y podemos decir adios a nuestro carro y a esta practica.



Conclusión

Durante la practica ahondamos mas en la estructura de los modelos 3D y especificamente cuales son los estandares o tipos de formatos que se usan en los software de modelado llamese .OBJ, .FBX, .GLTF, .DAE, etc. Ademas de realizar una investigacion previa, y luego confirmando dicha informacion de primera mano durante el laboratorio para entender que estos modelos efectivamente cargan con ellos la informacion necesaria (vertices, normales, Uvs, etc) para ser renderizados y en el caso de DAE, FBX o GLTF informacion extra o incluso texturas para diferentes casos, como animacion o texturizado.

Enseguida de esto exportamos modelos y utilizamos la librería Assimp para importarlos en nuestro proyecto de OpenGL. Esto confirmo el que estabamos trabajando con figuras no diferentes de las primitivas que utilizamos antes, y que nos permitio modificar de igual forma que hicimos con nuestra grua para poder simular un carro y la traccion de la llanta, la apertura del cofre y finalmente trasladar el modelo para que este pudiera avanzar o retroceder.

De manera que ahora podemos realizar tareas mas complejas y simular comportamientos emergentes en OpenGL.

Referencias:

- Apuntes de la clase de teoria y laboratorio de "Computacion Grafica e Interaccion Humano Computadora"